

2. 예측된 박스 및 클래스의 정확도 평가 방법

1 객체 모델 평가 정리

객체 탐지 모델의 성능을 평가하려면 정답 라벨(Ground Truth)과 모델의 예측값(Prediction)이 필요합니다.

- Ground Truth (GT): 모델이 찾아야 하는 실제 객체의 위치와 종류를 담고 있는 정답 파일
YOLO 형식의 .txt 파일은 각 객체의 클래스, 중심 좌표(x, y), 너비, 높이를 정규화된 값으로 저장하고, COCO JSON 형식은 더 복잡한 메타데이터와 함께 바운딩 박스 좌표, 세그멘테이션 마스크 등을 포함합니다.
- Prediction: 모델이 이미지에 대해 예측한 결과
이 결과는 보통 바운딩 박스(Bounding Box), 신뢰도 점수(Confidence Score), 그리고 클래스(Class)로 구성됩니다. 바운딩 박스는 객체가 있을 것으로 예상되는 직사각형 영역이고, 신뢰도 점수는 모델이 이 예측에 얼마나 확신하는지, 클래스는 어떤 객체인지 나타냅니다.
- 대표적인 평가 지표
 - IoU (Intersection over Union) 
보통 IoU가 특정 임계치(예: 0.5)를 넘어야만 예측이 '맞았다'고 간주됩니다.
 - Precision (정밀도)와 Recall (재현율)
이 두 지표는 모델이 얼마나 정확하게, 그리고 얼마나 많이 객체를 찾아냈는지를 상호 보완적으로 보여줍니다.

- True Positive (TP): 예측이 맞았고, 실제 정답도 있는 경우.
(예: 사과를 사과라고 정확히 예측함)
- False Positive (FP): 예측이 틀렸는데, 실제 정답은 없는 경우.
(예: 사과가 아닌 것을 사과라고 잘못 예측함)
- False Negative (FN): 예측을 못했는데, 실제 정답은 있는 경우.
(예: 실제 사과가 있는데 모델이 못 찾음)

○ 정밀도(Precision)는 모델이 '긍정'이라고 예측한 것들 중에서 실제로 '긍정'인 비율
예를 들어, 모델이 10개의 사과를 예측했는데 그 중 8개만 진짜 사과였다면 정밀도는 80%

$$Precision = \frac{TP}{TP + FP}$$

○ 재현율(Recall)은 실제 '긍정'인 것들 중에서 모델이 '긍정'이라고 예측한 비율
예를 들어, 이미지에 10개의 사과가 있었는데 모델이 7개만 찾았다면 재현율은 70%가 됨
놓친 객체가 얼마나 되는지를 보여주는 지표

$$Recall = \frac{TP}{TP + FN}$$

- **Average Precision (AP):** 하나의 클래스에 대해 여러 신뢰도 점수 임계치를 적용하며 정밀도-재현율 곡선을 그린 후 그 곡선 아래의 면적을 구한 값입니다. 이 면적이 넓을수록 성능이 좋음
- **mAP (mean Average Precision)**
mAP는 객체 탐지 모델의 최종 성능을 종합적으로 나타내는 가장 중요한 지표로서, 여러 IoU 임계치와 클래스에 대한 정밀도-재현율 곡선을 종합적으로 평가한 AP를

평균합니다.

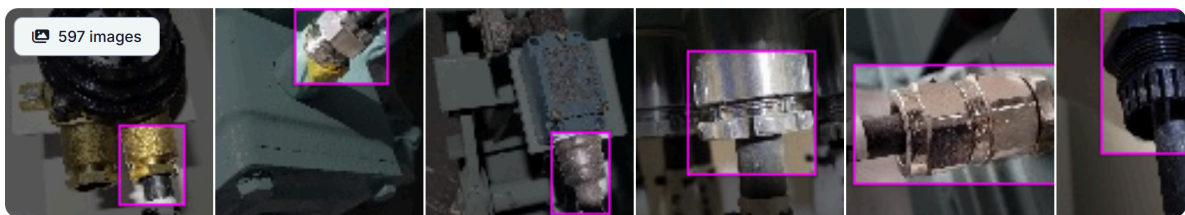
mAP 뒤에 붙는 숫자는 평가 기준이 되는 IoU 임계치를 의미

mAP@0.5: IoU 임계치를 0.5로 설정했을 때의 mAP 값을 뜻합니다.

mAP@0.5:0.95: IoU 임계치를 0.5부터 0.95까지 0.05 단위로 올리면서 측정한 mAP 값들을 모두 평균한 것입니다. 이 지표는 모델이 다양한 IoU 기준에 대해 얼마나 강인하게 성능을 내는지를 보여주므로, mAP@0.5보다 더 엄격하고 종합적인 평가 지표로 사용됩니다

2 올로를 이용한 불량탐지 & 성능평가

- <https://universe.roboflow.com/seok-6suvt/-nsokw>



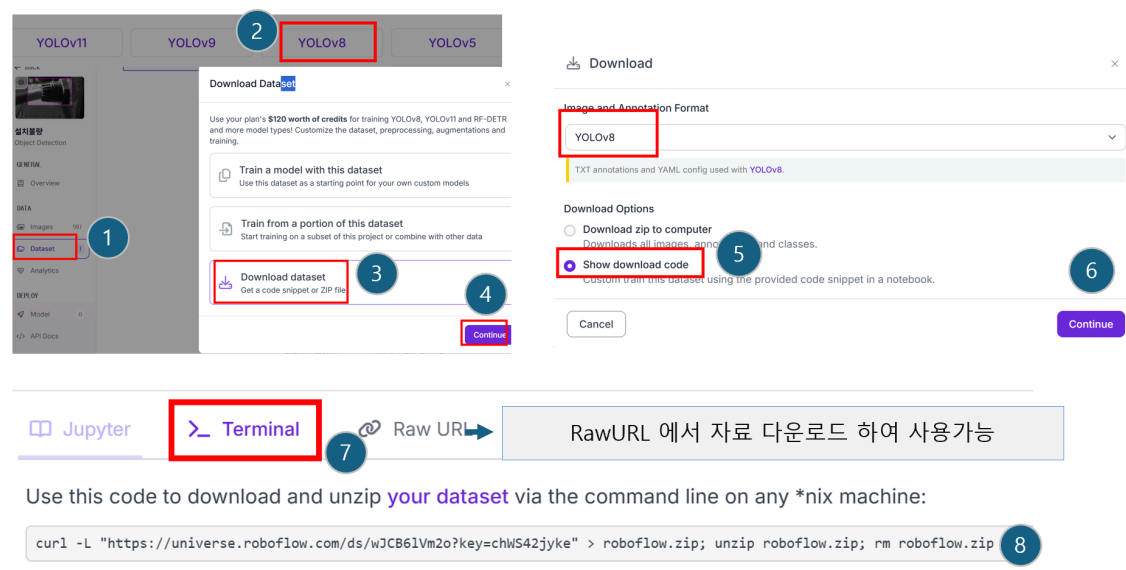
설치불량 Computer Vision Dataset



☆ 0 stars

Use this Dataset ▾

- 데이터셋 경로복사하여 다운로드 또는 자료다운로드하여 사용



Download Dataset

Use your plan's \$120 worth of credits for training YOLOv8, YOLOv11 and RF-DETR and more model types! Customize the dataset, preprocessing, augmentations and training.

Train a model with this dataset
Use this dataset as a starting point for your own custom models

Train from a portion of this dataset
Start training on a subset of this project or combine with other data

Download dataset
Get a code snippet or ZIP file

Continue

Download

Image and Annotation Format
YOLOv8
TXT annotations and YAML config used with YOLOv8.

Download Options
Download zip to computer
Download all images, annotations and classes.
Show download code
Custom train this dataset using the provided code snippet in a notebook.

Cancel Continue

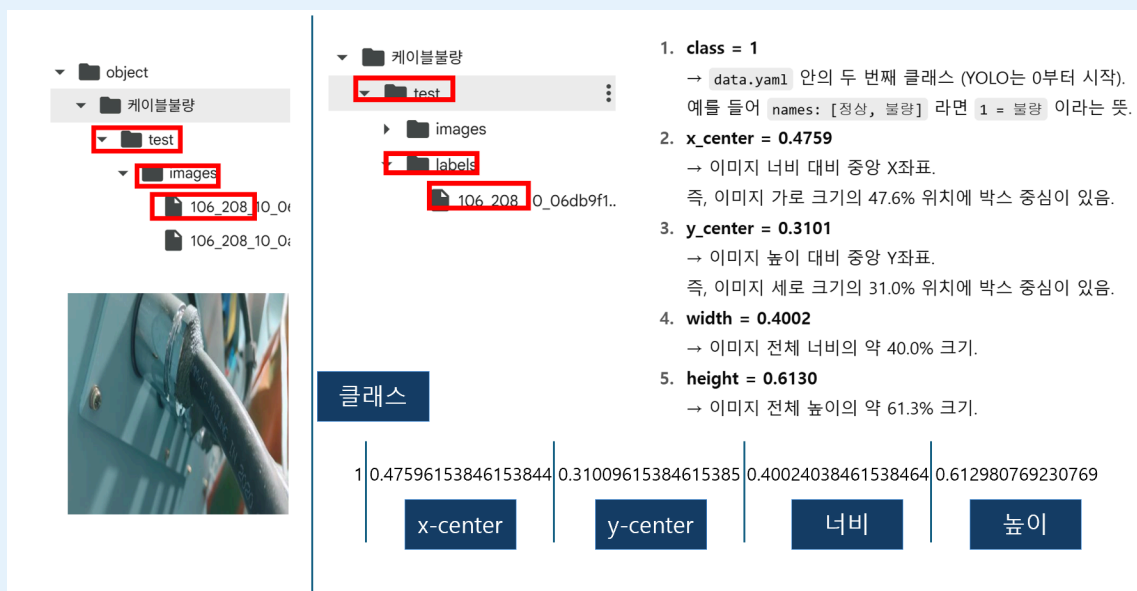
Jupyter Terminal Raw URL

RawURL 에서 자료 다운로드 하여 사용가능

Use this code to download and unzip your dataset via the command line on any *nix machine:

```
curl -L "https://universe.roboflow.com/ds/wJCB61Vm2o?key=chWS42jyke" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```

- 샘플이미지 및 label값확인



object

케이블볼량

test

images

106_208_10_0

106_208_10_0

케이블볼량

test

images

labels

106_208_0_06db9f1..

클래스

1. class = 1
→ data.yaml 안의 두 번째 클래스 (YOLO는 0부터 시작).
예를 들어 names: [정삼, 볼량] 라면 1 = 볼량 이라는 뜻.

2. x_center = 0.4759
→ 이미지 너비 대비 중앙 X좌표.
즉, 이미지 가로 크기의 47.6% 위치에 박스 중심이 있음.

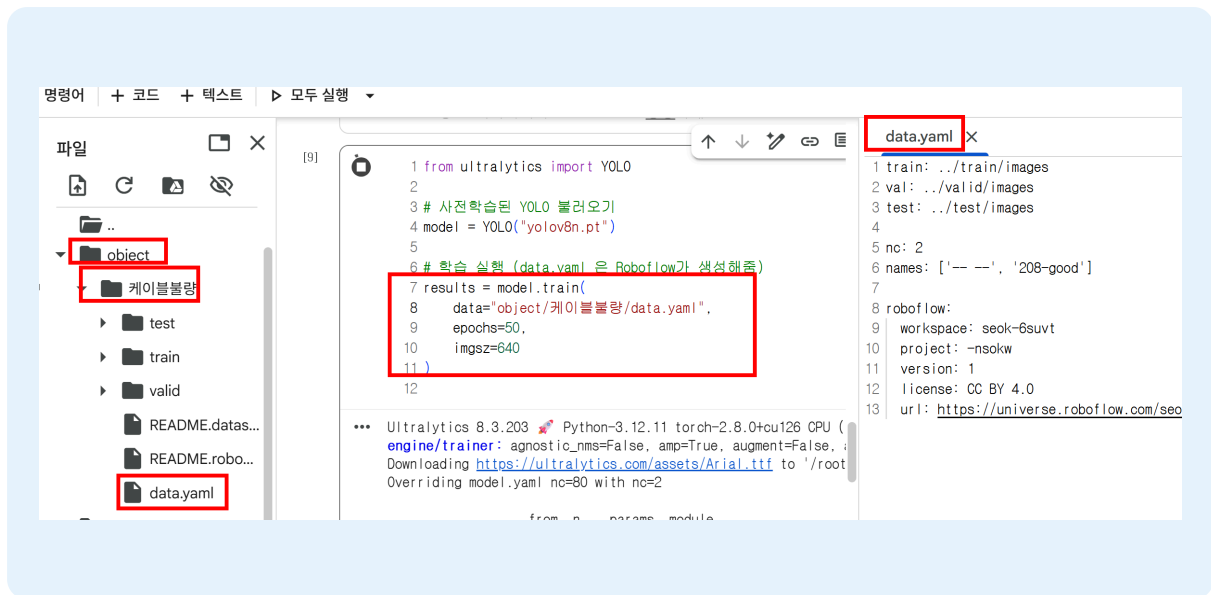
3. y_center = 0.3101
→ 이미지 높이 대비 중앙 Y좌표.
즉, 이미지 세로 크기의 31.0% 위치에 박스 중심이 있음.

4. width = 0.4002
→ 이미지 전체 너비의 약 40.0% 크기.

5. height = 0.6130
→ 이미지 전체 높이의 약 61.3% 크기.

1	0.47596153846153844	0.31009615384615385	0.40024038461538464	0.612980769230769
	x-center	y-center	너비	높이

- yaml 파일정보를 이용한 train



4일차(목)의 내용(2번)입니다. 옴로론은 훈련하면 자동으로 클래스 정확도를 계산해주기 때문에 IOU에 대한 개념만 알면 됩니다. - 캐글,구글검색,git 등에서 옴로론으로 할수 있는 작업 검색 또는 GPT와 함께 아이디어를 생각해 보세요.

📌 훈련결과

- 각 Epoch에서 세 가지 손실값이 나옵니다:

- **box_loss** : 바운딩 박스 위치 오차
- **cls_loss** : 클래스 분류 오차
- **dfl_loss (Distribution Focal Loss)** : 박스 좌표 정밀도 개선용 손실

👉 값이 줄어드는 게 정상이고, Epoch 2 → 5로 갈수록 **box_loss 1.59 → 1.41**, **cls_loss 2.77 → 2.04**, **dfl_loss 1.76 → 1.57** 로 조금씩 감소하고 있음

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
2/5	2.68G	1.59	2.773	1.764	31	640: 100%	68/68	12.3it/s	5.5s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 8.9it/s 0.4s
	all	118	118	0.156	0.309	0.148	0.0474		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
3/5	2.69G	1.578	2.51	1.73	26	640: 100%	68/68	12.9it/s	5.3s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 9.2it/s 0.4s
	all	118	118	0.219	0.371	0.199	0.0728		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
4/5	2.71G	1.529	2.292	1.675	18	640: 100%	68/68	12.3it/s	5.5s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 9.2it/s 0.4s
	all	118	118	0.258	0.563	0.32	0.146		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
5/5	2.73G	1.414	2.045	1.573	25	640: 100%	68/68	12.4it/s	5.5s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	4/4 9.2it/s 0.4s
	all	118	118	0.475	0.594	0.531	0.254		

5 epochs completed in 0.010 hours.
Optimizer stripped from /content/runs/detect/train/weights/last.pt, 6.2MB
Optimizer stripped from /content/runs/detect/train/weights/best.pt, 6.2MB

정밀도(p) 재현율(R)

Validating /content/runs/detect/train/weights/best.pt...

Ultralytics 8.3.203 Python-3.12.11 torch-2.8.0+cu126 CUDA:0 (NVIDIA A100-SXM4-40GB, 40507MiB)
Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):
all	118	118	0.476	0.594	0.531	0.256
---	60	60	0.44	0.7	0.53	0.239
208-good	58	58	0.512	0.488	0.532	0.274

Speed: 0.1ms preprocess, 0.5ms inference, 0.0ms loss, 1.0ms postprocess per image
Results saved to /content/runs/detect/train

성능지표

- 성능 지표 (Validation)

- **Precision (정밀도)** : Epoch 2는 15.6% → Epoch 5는 47.5%까지 향상(예측 50% 맞춤)
- **Recall (재현율)** : Epoch 2는 30.9% → Epoch 5는 59.4%까지 향상
- **mAP@0.5** : Epoch 2는 0.148 → Epoch 5는 0.531 (50% IoU 기준, 53% 정확도)
- **mAP@0.5:0.95** : Epoch 2는 0.047 → Epoch 5는 0.254 (더 엄격한 IoU 기준에서도 개선됨)

- test데이터셋으로 성능평가

```

1 # test 세트로 평가
2 metrics = model.val(data="object/케이블불량/data.yaml", split="test")
3 print("mAP50:", metrics.box.map50)
4 print("mAP50-95:", metrics.box.map)
5 print("Precision:", metrics.box.mp)
6 print("Recall:", metrics.box.mr)
7

```

```

Ultralytics 8.3.203 Python-3.12.11 torch-2.8.0+cu126 CUDA:0 (NVIDIA A100-SXM4-40GB, 40507MiB)
val: Fast image access (ping: 0.0±0.0 ms, read: 635.2±230.1 MB/s, size: 16.4 KB)
val: Scanning /content/object/케이블불량/test/labels... 60 images, 0 backgrounds, 0 corrupt: 100%
val: New cache created: /content/object/케이블불량/test/labels.cache

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	100%
all	60	60	0.39	0.602	0.525	0.256	4/4
-- --	30	30	0.359	0.767	0.604	0.286	
208-good	30	30	0.421	0.437	0.446	0.225	

```

Speed: 3.4ms preprocess, 9.2ms inference, 0.0ms loss, 3.2ms postprocess per image
Results saved to /content/runs/detect/train3
mAP50: 0.5252525559748633
mAP50-95: 0.25569148995750035
Precision: 0.39003666291098804
Recall: 0.6018499405126783

```

- test이미지로 확인

```

file_path = "/content/object/케이블불량/test/images/106_208_10_06db9f18-4
173-40e5-8058-3ce16ca29082_jpg.rf.db0d1664dc846dc0cc663c2adf32c
371.jpg"
# 예측 실행
results = model.predict(file_path, conf=0.25)
# 내장된 show() → 박스 + 클래스 + confidence 자동 표시
results[0].show()

```

image 1/1 /content/object/케이블불량/test/images/106_208_10_06d
Speed: 2.7ms preprocess, 7.3ms inference, 1.4ms postprocess per

